Reinforcement Learning based Module Placement for Enhancing Reliability of MEDA Digital Microfluidic Biochips

Debraj Kundu
IIT Roorkee, India
dkundu@cs.iitr.ac.in

Gadikoyila Satya Vamsi
IIT Roorkee, India
gvamsi@cs.iitr.ac.in

Karnati Vivek Veman
IIT Roorkee, India
kveman@cs.iitr.ac.in

Gurram Mahidhar
IIT Roorkee, India
gmahidhar@cs.iitr.ac.in

Sudip Roy
IIT Roorkee, India
sudip.roy@cs.iitr.ac.in

ABSTRACT

Micro-electrode-dot-array (MEDA) based biochips are one of the promising new generation microfluidic biochips consisting of a sea-of-micro-electrodes with dedicated detection circuit for each micro-electrode. Moreover, the ability to manipulate discrete droplets of different volumes and to route them in any direction presents MEDA biochips as an advanced microfluidic technology. Due to similarity in the working principles, the reliability issues of both MEDA biochips and digital microfluidic biochips are similar. In this paper, we propose a module placement technique for MEDA biochips to improve the reliability of biochips. *Reinforcement learning based placement method* (*RLPM*) is designed for obtaining the reliability-aware placement of rectilinear shaped microfluidic modules. *RLPM* aims to minimize the area of a biochip while increasing its reliability. Simulation results confirm that on average *RLPM* minimizes the chip utilization area by 28.6% while enhancing the reliability of MEDA biochips compared to the state-of-the-art method.

CCS CONCEPTS

• **Hardware** → **Electronic design automation**; **Emerging technologies**.

KEYWORDS

microfluidics, biochips, MEDA, DQN, placement, reliability

ACM Reference Format:
Debraj Kundu, Gadikoyila Satya Vamsi, Karnati Vivek Veman, Gurram Mahidhar, and Sudip Roy. 2023. Reinforcement Learning based Module Placement for Enhancing Reliability of MEDA Digital Microfluidic Biochips. In *Proceedings of the Great Lakes Symposium on VLSI 2023 (GLSVLSI '23), June 5–7, 2023, Knoxville, TN, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3583781.3590209

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GLSVLSI '23, June 5–7, 2023, Knoxville, TN, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0125-2/23/06…$15.00
https://doi.org/10.1145/3583781.3590209

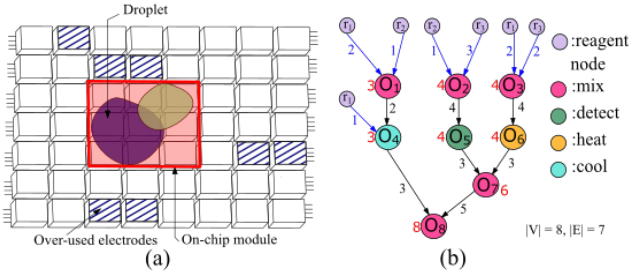


Figure 1: (a) Top-view of a typical MEDA chip layout [6] and (b) a sequencing graph representing a bioprotocol.

1 INTRODUCTION

Microfluidic biochip ias one of the most promising platforms for realizing various bioprotocols like prenatal screenings [13], drug discovery [3], point-of-care medical diagnostics [12], etc.

Apart from the cost effectiveness and portability of these microfluidic biochips, they also possess a bunch of advantages over lab-based macrosystems in terms of automation, accuracy, precision and parallelization of different biochemical operations. In view of the global crisis due to the COVID-19 pandemic, microfluidic biochips are also being used for the detection of Coronavirus [11]. In the last decade, advancement of the fabrication technologies for both digital microfluidic biochips (DMFBs) [6, 14] and continuous-flow microfluidic biochips (CFMBs) [5] opens up a new horizon for research in this domain. A micro-electrode-dot-array (MEDA) biochip [14] is an advanced DMFB chip, which consists of an array-of-micro-electrodes as shown in Fig. 1(a), where each micro-electrode is integrated with a dedicated sensor and activation circuit. This makes MEDA biochips more flexible and error-resilient than conventional DMFBs. Moreover, a MEDA biochip can realize rectilinear-shaped on-chip devices (referred as modules) and any-angle routing of fluid droplets with different volumes. So in this work, we consider both rectangular and "L"-shaped modules for the synthesis of MEDA biochips. As an "L"-shaped mixer module is realized as a union of two rectangular modules of sizes $h \times w$ and $h' \times w'$, respectively, we use the notation $h \times w|h' \times w'$ to denote an "L"-shaped module. The working principle of both MEDA and conventional DMFBs for the manipulation of the droplets are based on electro-wetting-on-dielectric (EWOD) mechanism [14]. Although a MEDA biochip has many advantages over conventional DMFBs, it still suffers from electrode degradation due to over actuations of the micro-electrodes. One on-chip module configured on a MEDA chip is shown in Fig. 1(a), where some over-used micro-electrodes are shaded. It means that any further operation on these over-used

micro-electrodes may jeopardize the reliability of the fluidic operation performed on the chip.

A synthesis methodology is required to design a MEDA chip layout in order to automate the execution of a bioprotocol on it. A bioprotocol is generally represented as a sequencing graph with two types of nodes namely input reagent nodes r_is and operation nodes O_is (Fig. 1(b)). Due to the architectural differences of various microfluidic platforms (e.g., DMFBs, CFMBs, etc.), we need an architecture specific synthesis methodology to efficiently realize any bioprotocol. The synthesis of MEDA biochips involves different phases like scheduling of fluidic operations, fluidic module binding and placement, fluid droplet routing, etc. and to date several approaches have been proposed for one or more such phases [4, 8, 15]. However, the *high-level synthesis for MEDA* (*HSM*) [8] and the *wear-leveling based synthesis for MEDA* (*WLSM*) [15] are the only two work to present the entire synthesis methodology for MEDA biochips considering only rectangular modules. *HSM* [8] takes no measure to control the reliability of a MEDA biochip, while *WLSM* [15] considers reliability improvement as the primary objective. However, none of them aim to minimize the chip utilization area for obtaining the smallest size MEDA biochip and neither of them considers the rectilinear shaped modules for synthesis of it.

Given a bioprotocol, we propose a placement technique considering the rectilinear shaped modules that minimizes the chip utilization area of a MEDA biochip while maintaining the reliability of the chip. We present a Deep-Q-Network (DQN) [10] based reinforcement learning (RL) approach called *reinforcement learning based module placement method* (*RLPM*) to simultaneously improve the reliability and minimize the chip utilization area while determining the module placement and schedule on the MEDA biochip.

The remainder of the paper is organized as follows. Sec. 2 describes the prior work and motivation. In Sec. 3, the preliminaries and the overview of the work are illustrated. Sec. 4 describes the proposed and Sec. 5 presents the simulation results. Finally, Sec. 6 concludes the paper.

2 PRIOR WORK AND MOTIVATION

2.1 Prior Work

As MEDA biochip is the next generation DMFBs with advanced functionalities, several research articles on its design automation were published during the last decade. Li et al. [8] proposed a method called *high-level synthesis for MEDA* (*HSM*) that targets the co-optimization of operation scheduling, module placement, and droplet routing. Since reliability is one of the primary concerns of any biochip, Zhong et al. [15] recently presented a reliability-aware synthesis methodology called *wear-leveling based synthesis for MEDA* (*WLSM*). Further, Chung et al. [1] proposed a module placement algorithm for MEDA biochip that incorporates the completion-time uncertainties of on-chip operations. In summary, researchers have been working towards improving the reliability of the MEDA biochips by proposing efficient techniques for different phases of MEDA biochip synthesis.

2.2 Motivation

An experimental study by Dong et al. [2] inferred that an electrode in a DMFB is completely degraded after it is actuated approximately over 200 times due to charge trapping. As a result, the force to move a droplet sitting on the degraded electrode decreases and the droplet is likely to get stuck on that charge-trapped electrode. Moreover, excessive actuation of a set of electrodes can also lead to dielectric breakdown, resulting in the malfunctioning of on-chip fluidic operations. The electrodes configured as a module are likely to be actuated more frequently than other electrodes which are used only for droplet routing. So the reliability of a MEDA biochip is largely dependent on an effective placement of the modules on the chip. On the other hand, the chip utilization area is another important parameter, which needs to be minimized in order to design small-sized portable microfluidic biochip. With a larger chip area it is easier to place the modules without much overlapping among themselves, which in turn increases the reliability of the chip. Hence, the reliability increases if chip utilization area increases. Practically, it is desired to minimize the chip utilization area, whereas at the same time reliability of the chip should be high. Note that none of the prior works has considered these two opposite objectives simultaneously. This motivates us to find an efficient placement method for MEDA biochips that can increase the reliability of the chip while minimizing the chip utilization area.

3 PRELIMINARIES AND OVERVIEW

3.1 Preliminaries: Modeling of MEDA Biochip

Without loss of generality, we model a MEDA biochip of height H and width W as a 2-D grid $H \times W$, where each grid-cell represents a micro-electrode.

In order to accurately model the synthesis problem for MEDA biochips, two unique properties of MEDA are considered: (i) manipulation of droplets of different volumes and (ii) droplet shape changing ability, which includes both shape morphing and splitting. Due to the limitation of the EWOD force we have to consider an upper bound of the droplet volume d. Hence, in this work, the droplet volume d is chosen to be in the range of $[1, 8]$. When $d = 1$, it is referred as an unit-droplet, which occupies the space of a unit-cell in such a way that no other droplet can reside in that unit-cell. In this work the size of an unit-cell is assumed to be 2×2 consisting of 4 micro-electrodes and the volume of an unit-droplet ($d = 1$) is considered as the capacity of an unit-cell. For any larger volume, i.e., $d > 1$, we generalize the shape of the droplet as a multiple of unit-cells referred as reference-block $h \times w$. For instance if $2 \leq d \leq 4$, then reference block is assumed to be of 2×2 unit-cells and if $5 \leq d \leq 8$, then it is assumed to be of 3×3 unit-cells. All the above assumptions are considered, when the droplets are in normal shapes, i.e., without considering the changed shapes of the droplets. At first, the shape of a unit-droplet can neither be changed nor it can be split into multiple smaller droplets, while any droplet of volume $d > 1$ can be morphed or spit into multiple smaller droplets. If $2 \leq d \leq 4$, then it can either be morphed from 2×2 unit-cells to $1 \times d$ unit-cells or it can be split into d unit-droplets. However, if $5 \leq d \leq 8$, then the droplet shape can either be morphed from 3×3 unit-cells to $2 \times \lceil \frac{d}{2} \rceil$ unit-cells or can be equally split into two droplets of 2×2 unit-cells.

Modules (devices) are configured using a set of electrodes on the biochip. As shown in Table 1, we consider a typical module library $\mathcal{M}$ having eight different types of modules out of which

the first five are mixer modules and the last three modules are for detection, heating and cooling operations, respectively. In Table 1, the second column denotes the dimensions (sizes) of the modules in terms of $h \times w$ unit-cells, and the fourth column represents the total unit volume of the fluid (capacity) that a module can handle for its operation. The operation time (the last column in Table 1) for fluid mixing or detection depends on the module on which it is executed, whereas for heating and cooling the operation times are predefined in the sequencing graph of the bioprotocol.

3.2 Overview of the Synthesis Approach

The whole synthesis approach involves different phases like module binding, scheduling, and placement.

3.2.1 Module Binding. For a given sequencing graph $G(V, E)$, we use a static module binding strategy to bind the modules with each of the operation nodes in G. In this work, we have chosen the sequencing graphs with four different types of operations: (i) mix, (ii) heat, (iii) cool, and (iv) detect, and a module library as shown in Table 1. We choose an iterative strategy for module binding, where at each iteration we select all the leaf operation nodes O_is, bind a suitable module M_j (from Table 1) with each of them, and then remove those O_is from G. In order to bind the modules from Table 1 we define the following selection rules. For all the mix operations with input size less than or equals to 4, we fix the selection priority of the modules as $< M_3, M_2, M_5, M_1 >$ and if the input size is greater than 4, then by default we have to select M_4. We have also assumed that except M_1 and M_4, at most two consecutive instances of M_2, M_3, and M_5 can be used for binding. After the module binding we label each O_i with corresponding (O_i, M_j, T_i) tag, where M_j is a module of type j and T_i is the operation time for O_i. The module binding information $\widehat{B} = \bigcup_{\forall i \in V}(O_i, M_j, T_i)$ will be used in the successive schedule and placement. During the placement of a module M_j we also need its area information, which is calculated as the total number of cells occupied by the module and denoted as $|M_j|$.

3.2.2 Scheduling. Before module placement on $H \times W$ grid, for scheduling the operation nodes we build two lists namely active list ($\mathcal{L}_1$) and ready list ($\mathcal{L}_2$). Initially $\mathcal{L}_1$ is kept empty, but to fill $\mathcal{L}_2$ we select all the leaf nodes (O_is $\in V$) of G to calculate their priorities by the equation, $S(O_i) = \alpha_1 \times \frac{1}{(\tilde{h} - \ell_i)} + \alpha_2 \times \frac{1}{|M_j|} + \alpha_3 \times \frac{1}{T_i}$, where $\tilde{h}$ and ℓ_i are the height of G and the level of O_i in G, respectively, and α_1, α_2 and α_3 are the parameters for the weights or importance to set the priorities of O_is, such that $0 \leq \alpha_1, \alpha_2, \alpha_3 \leq 1$ and $\alpha_1 + \alpha_2 + \alpha_3 = 1$. In order to place multiple modules simultaneously, at a particular depth of G we prioritize the O_is that can be bound with the modules of smaller size ($|M_j|$) and smaller operation time (T_j). Moreover, we also want to prioritize the O_is at lower depths of G. For instance, in this work we consider $\alpha_1 = 0.4, \alpha_1 = 0.3$ and $\alpha = 0.3$. Then we insert the respective module binding information $(O_i, M_j, T_i) \in \widehat{B}$ into $\mathcal{L}_2$ in a descending order of the values of $S(O_i)$s. Next, after the module for an operation O_i is placed by the proposed method (*reinforcement learning based placement method*, i.e., *RLPM*), the corresponding $(O_i, M_j, T_i) \in \widehat{M}$ is removed from the list $\mathcal{L}_2$ and insert it in $\mathcal{L}_1$. Simultaneously $\mathcal{L}_2$ is also updated with all the module binding information of the child nodes of O_i.

Table 1: A typical module library for MEDA chip synthesis.

Module M_j	Size $h \times w$	Module Name	Capacity (unit vol.)	Operation time
M_1	2×2	Mixer: circular	2	10
M_2	2×4	Mixer: circular	4	7
M_3	3×3	Mixer: SAR-laminar	4	3
M_4	4×4	Mixer: SAR-laminar	8	4
M_5	$4 \times 2 \vert 2 \times 2$	Mixer: L-circular	4	5
M_6	3×3	detector	8	1
M_7	3×3	heater	8	T_h †
M_8	3×3	cooler	8	T_c †

† T_h and T_c are heating time and cooling time as per the bioprotocol.

3.2.3 Placement. In the placement phase when a module is placed on a MEDA biochip due to reconfigurability nature of MEDA some of the micro-electrodes may be over-utilized, which degrades the reliability of the MEDA chip. So, with the aim to generate a reliable synthesis methodology we introduce a parameter called overlapping constraint $\alpha \in \mathbb{Z}^+$. For a given MEDA biochip of $H \times W$ size, the overlapping constraint α implies that a micro-electrode can be overlapped by at most α number of modules. Maintaining the given overlapping constraint of a MEDA biochip of $H \times W$ size, *RLPM* returns the placement and scheduling information $\widehat{P}$ for any O_i as $Place(O_i) = < |M_j|, (x_i, y_i), ts_i, te_i >$, where (x_i, y_i) is the top-left coordinate of the placed module M_j on the $H \times W$ grid, where $ts_i and te_i$ are the start and end time-stamps of O_i, respectively, such that $T_i = te_i - ts_i$.

4 REINFORCEMENT LEARNING BASED PLACEMENT METHOD: *RLPM*

After obtaining the module binding information $\widehat{B}$ and initializing the active list $\mathcal{L}_1$ and ready list $\mathcal{L}_2$, we build a Deep-Q-Network (DQN) based method called *reinforcement learning based placement method* (*RLPM*), which efficiently places the modules on the MEDA biochip. The problem formulation for *RLPM* is as follows:

Inputs: (1) A sequencing graph $G(V, E)$, a module library $\mathcal{M}$, and the module binding information $\widehat{B} = \bigcup_{\forall O_i \in V}(O_i, M_j, T_i)$, (2) the active list $\mathcal{L}_1$ and the ready list $\mathcal{L}_2$, and (3) a MEDA biochip of size $H \times W$ with an overlapping constraint α.

Outputs: (1) Placement and scheduling information $\widehat{P} = \bigcup_{\forall i \in V} Place(O_i)$, where $Place(O_i) = < M_j, (x_i, y_i), ts_i, te_i >$, (2) the total number of utilized cells, and (3) the bounding-box area $\mathcal{B}$.

Objectives: While maintaining the given overlapping constraint α the primary objectives are: (1) to minimize the total place-and-schedule time (2) to minimize the bounding-box area $\mathcal{B}$, or (3) to maximize the degree of reliability ρ (defined later).

4.1 Modeling for *RLPM*

The modeling of the *RLPM* is explained through building of the MEDA environment and the state, action and reward used in *RLPM*.

4.1.1 Building MEDA Environment. We consider the MEDA biochip layout as $H \times W$ grid to define the MEDA environment. For the placement of module M_j to perform the operation O_i, the placement and scheduling information is written as $Place(O_i) = < M_j, (x_i, y_i), ts_i, te_i >$. Here the set of utilized cells for the placed module M_j is denoted by $\mathcal{A}_i$ referred as the set of active-cells during the time-stamp range (ts_i, te_i). Each module M_j is defined as an *agent*, which explores the MEDA environment to decide its placement. Moreover, to maintain the reliability of the MEDA biochip, we define the overlapping constraint, α, which signifies the upper

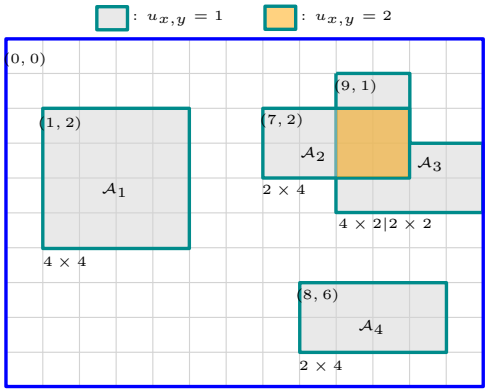


Figure 2: A MEDA environment with four obstacles ($\mathcal{A}_1$, $\mathcal{A}_2$, $\mathcal{A}_3$, $\mathcal{A}_4$) showing different $u_{x,y}$ values.

limit of the utilization of micro-electrode by different modules. So, $\alpha = 0$ implies that no operation can be implemented (as no module can be placed) on the biochip, which is impractical. Again $\alpha = 1$ or $\alpha = 2$ means each micro-electrode can be used at most by one or two modules, respectively. Further, we define an overlapping value for each micro-electrode denoted by $u_{x,y}$, which must be within the range $[0, \alpha]$, where $u_{x,y}$ corresponds to the electrode at position (x, y) on $H \times W$ grid. So the placement constraints for *RLPM* are: (i) modules should always be placed within the $H \times W$ grid, (ii) no active-cell should be utilized by more than one module, and (iii) $\forall(x, y), 0 \leq u_{x,y} \leq \alpha$. Based on these placement constraints at any time-stamp t, the regions of the MEDA environment where any other module cannot be placed, i.e., $\cup_{\forall i}\mathcal{A}_i$, are called "obstacles" as shown in Fig. 2. The cells with $u_{x,y} = \alpha$, are considered as permanent "obstacles", as depicted in Fig. 2 with yellow color.

4.1.2 State. Whenever an operation O_i ends at time-stamp t two tasks are performed: (i) the corresponding micro-electrodes in $\mathcal{A}_i$ are freed, and (ii) the ready list $\mathcal{L}_2$ is updated with the module binding information $\widehat{B}$ of child operations of O_i. Let (O_p, M_j, T_p) be the next element in the ready list $\mathcal{L}_2$ for which M_j is placed at the same time-stamp t, then after its placement, the MEDA environment will change and the *state* is designed in such a way that it reflects these situations of the MEDA environment at different time-stamps. A brute-force method to define a *state* is to consider an image of the entire MEDA environment at a particular time-stamp. However, in order to keep the problem simple we have considered a *state* as $s_i = < \mathcal{B}, x, y >$, where $\mathcal{B}$ is the current bounding-box area and x, y is the position of the last module placed on the MEDA environment.

4.1.3 Action. In order to find a suitable place for an agent M_j, all the maximal rectangles are determined on the current MEDA environment excluding the "obstacles" at time-stamp t. Further, taking the union of two or more maximal rectangles which have some common boundaries, we can determine the "maximal rectilinear polygons" in polynomial time. Then all possible locations (top-left coordinates) inside those "maximal rectilinear polygons" are estimated, which are referred as "candidate positions" to place an M_j. For a given MEDA environment, we also rotate thrice by 90° and check all possible rectilinear orientations of any module M_j within those "maximal rectilinear polygons". The number of rotations required to find any "candidate position" is symbolized as φ, where

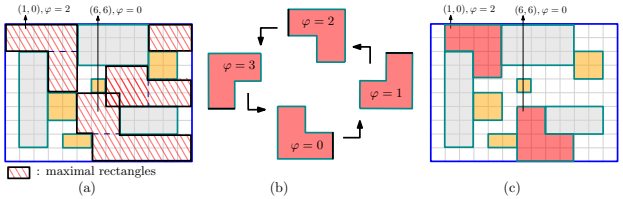


Figure 3: (a) The "maximal rectilinear area" for a given number of obstacles. (b) All possible rectilinear orientations of an "L"-shaped module, and (c) possible positions for placement of this module in that "maximal rectilinear area".

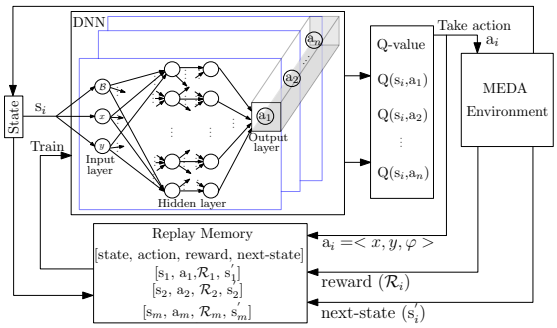


Figure 4: DQN modeling for *RLPM*.

$\varphi \in \mathbb{Z}^+$ and $0 \leq \varphi \leq 3$ (e.g., Fig. 3(b)). In the modeling for *RLPM*, *actions* are represented as $a_i =< x, y, \varphi >$, where (x, y) denotes the "candidate position" and φ implies the number of rotations required for the module (*agent*) to be placed at (x, y). For a given set of "obstacles" at time-stamp t, all the desired maximal rectangles are shown in Fig. 3(a). For instance, if M_j of size $4 \times 2|2 \times 2$ needs to be place on the MEDA environment at t, then all possible rectilinear orientations of M_j are shown in Fig 3(b). As shown in Fig. 3(c), there are two possible positions: (i) $(1, 0)$ with two rotations, i.e., $\varphi = 2$, and (ii) $(6, 6)$ with no rotation, i.e., $\varphi = 0$. So, the possible *actions* for the *agent* corresponding to the module $4 \times 2|2 \times 2$ are $< 1, 0, 2 >$ and $< 6, 6, 0 >$. As the set of "obstacles" varies with time, the possible set of *actions* are also dynamically changing for each *state*. So, we consider a deep-neural-network (DNN) as shown in Fig. 4, where the input layer, i.e., the state-layer has 3 neurons with respect to $< \mathcal{B}, x, y >$, the output layer, i.e., the action-layer has one neuron, and in between it has two hidden layers with 256 and 128 neurons, respectively. For the *current-state* $< \mathcal{B}, x, y >$ and an *agent* corresponding to the module M_j, at first we determine all possible *actions* and consider the DNN model for each possible *action*. Then for each *current-state* and *action* pair, the DNN predicts the corresponding Q-values. Next, we select the action corresponding to the maximum Q-values and update the *current-state* with the *next-state* corresponding to the selected *action* and repeat the same process whenever a new module needs to be placed.

4.1.4 Reward. After determining the *next-state* from the chosen *action*, we need to determine a reward to grade the *action*. The reward function is formulated in such a way that it encapsulates the parameters which ultimately help in achieving our objectives.

The 1st objective is to minimize the total place-and-schedule time $\mathcal{T}_p$, which is primarily dependent on the total number of scheduling steps. This time $\mathcal{T}_p$ can be reduced by maximizing the placement of modules with respect to the operations in the ready list $\mathcal{L}_2$ (refer

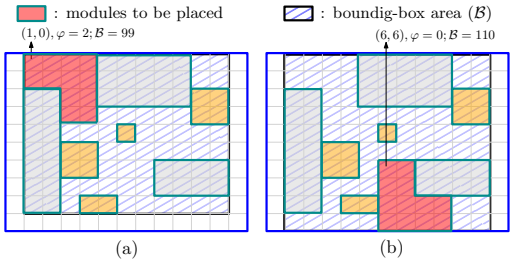


Figure 5: Estimation of $\mathcal{B}$ for two different actions (a) $< 1, 0, 2 >$ and (b) $< 6, 6, 0 >$.

Sec. 3.2.2) at every time-stamp. In general, if a module is placed towards the center of a MEDA biochip it restricts the possibilities for placement of next modules. As a result multiple modules cannot be placed simultaneously at same time-stamp. Hence, to achieve the 1st objective, we defined the function $\mathcal{F}_1(H, x, y) = \frac{H}{(x^2+y^2+1)}$. This function increases when a module is placed towards the top-left corner of the chip, i.e., the cell $(0, 0)$. Next, to achieve the 2nd objective, i.e., to minimize the bounding box area ($\mathcal{B}$) of the utilized cells, we consider two positive integers μ_1 and μ_2, such that $\mu_1 > \mu_2$ and $\mu_1\mu_2 = \sum_{\forall i \in \widehat{B}} |M_j|$, where $\sum_{\forall i \in \widehat{B}} |M_j|$ is an estimation of the total module area determined from the module binding information $\widehat{B}$. The difference between $\mu_1\mu_2$ and $\mathcal{B}$ is increased effectively with the help of: (i) overlapping of the modules, maintaining the placement constraints and (ii) finding the most suitable location for the modules in "maximal rectilinear polygons". In Fig. 5, two different $\mathcal{B}$s are shown as the shaded regions for two separate actions. Clearly the action $< 1, 0, 2 >$ is better than $< 6, 6, 0 >$ as it minimize the $\mathcal{B}$. To minimize $\mathcal{B}$ we define the function $\mathcal{F}_2 = \mu_1 - \frac{2\times\mathcal{B}}{\mu_2}$, which returns larger values when $\mathcal{B}$ is smaller than $\mu_1\mu_2$. Finally, for the 3rd objective to achieve a routing-aware placement, we define a function $\mathcal{F}_3 = \sum_{\forall e_{i,j} \in E} C_{i,j}$, where $e_{i,j}$ is the edge between O_i and O_j in the sequencing graph. $C_{i,j}$ is the number of overlapping electrodes between $\mathcal{A}_i$ and $\mathcal{A}_j$, i.e., $|\mathcal{A}_i \cap \mathcal{A}_j|$, such that O_j is the parent node of O_i. This helps to minimize the distance between the placements of the parent and child operations. Finally, we divide the sum of all the $\mathcal{F}_i$s with $\sum_{\forall (x,y)} 2^{u_{x,y}-\alpha}$ to ensure a reliable placement. For each $\mathcal{F}_i$ we assign a positive integer coefficient ω_i, which helps to set-up the relative importance among three objective functions. Hence, the proposed reward function can be written as

$$\mathcal{R} = \left[\omega_1 \times \frac{H}{(x^2+y^2+1)} + \omega_2 \times (\mu_1 - \frac{2\times\mathcal{B}}{\mu_2}) + \omega_3 \times \sum_{\forall e_{i,p} \in E} C_{i,p}\right] \times \frac{1}{\sum_{\forall (x,y)} 2^{u_{x,y}-\alpha}}.$$

Based on the priorities of the three objectives, the values of ω_1, ω_2 and ω_3 need to be finalized. After some suitable iterations, when the model learns and optimizes (maximizes) the value of the reward function, we get the placement and scheduling information for each O_i as $Place(O_i) = < M_j, (x_i, y_i), ts_i, te_i >$ recorded in $\widehat{P}$.

4.2 Evaluation of the Placement Parameters

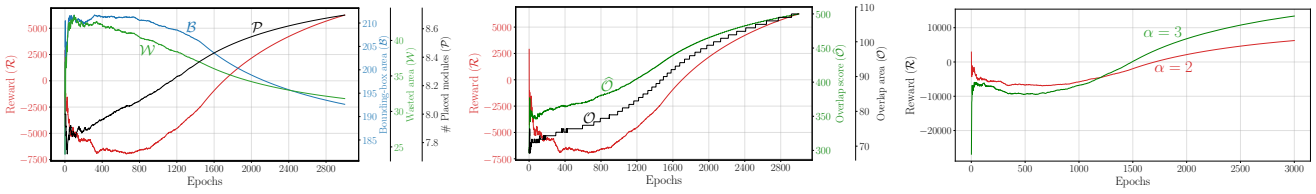
Given a sequence graph of a bioprotocol *RLPM* determines the placement and scheduling information stored in $\widehat{P}$ while maintaining the reliability of the MEDA biochip. In order to evaluate the performance of *RLPM*, we consider four parameters: (i) the total number of utilized cells (U), (ii) the bounding-box area ($\mathcal{B}$), and (iii) the degree of reliability (ρ). We can estimate U as $U = \left|\bigcup_{\forall i \in V} \mathcal{A}_i\right|$ and ρ as $\rho = \frac{U}{\sum_{\forall (x,y)|u_{x,y} \le \alpha} u_{x,y} + \sum_{\forall (x,y)|u_{x,y} > \alpha} u_{x,y}^2}$. The value of ρ is in the range of $(0, 1]$, and also ρ is inversely proportional to the degradation of the biochip. Once we estimate U, the bounding box $\mathcal{B}$ can be estimated as the minimum rectangle including all the cells in U.

5 SIMULATION RESULTS

All simulations are performed in a PC with a 3.70 GHz Intel Xeon processor and 32 GB RAM running 64-bit Windows 10. We have considered seven bioprotocols as our testcases, which includes both real-life and synthetically designed bioprotocols. The details of the bioprotocols taken as testcases are as follows: <testcase_ID, bioprotocol name, #operations, #edges>: <T1, Polymerase Chain Reaction (PCR), 7, 6> [9]; <T2, In-Vitro Diagnostics (IVD), 12, 6> [9]; <T3, Colorimetric Protein Assay (CPA), 55, 54> [9]; <T4, Synthetic Benchmark I, 10, 9>; <T5, Synthetic Benchmark II, 20, 19>; <T6, Synthetic Benchmark III, 30, 29>; <T7, Synthetic Benchmark IV, 9, 10> [7]. For the simulation purpose we consider the module library as mentioned in Table 1 and the size of the unit-cell is assumed as 2×2 micro-electrodes. For these seven testcases the upper and lower limits of the coefficients in the reward function are as follows, $1 \le \omega_1 \le 20, 1 \le \omega_2 \le \sum_{\forall i \in \widehat{B}} |M_j|$ and $1 \le \omega_3 \le 5$.

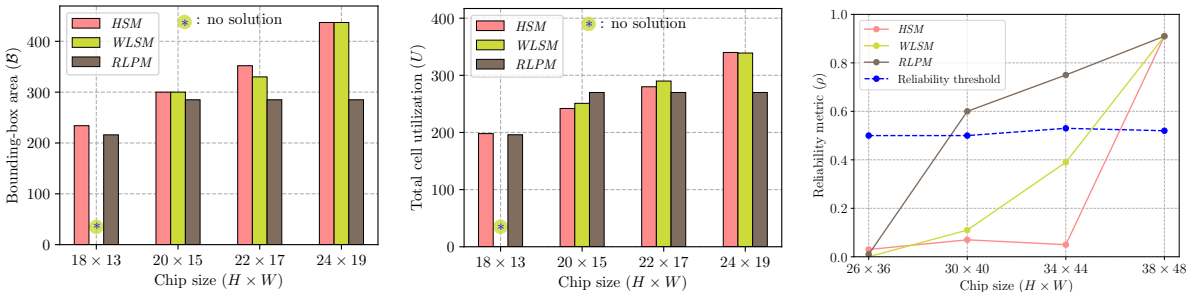
The validity and the performance of *RLPM* can be realized from the distribution of the variables mentioned in $\mathcal{F}_1$, $\mathcal{F}_2$ and $\mathcal{F}_3$ with respect to the distribution of the reward $\mathcal{R}$. In its initial epochs *RLPM* may determines some solution which fails to place all the modules, maintaining the overlapping constraint of the MEDA bichip α. So in order to check whether *RLPM* learns from those mistakes and able to increase the number of placed modules with more number of epochs, we consider a variable $\mathcal{P}$ to represent the number of placed modules at each epoch and the variable $\mathcal{W}$ to denote the wasted area, calculated as $\mathcal{W} = \mathcal{B} - U$. For testcase T6, Fig. 6(a) depicts the distribution of the average values of $\mathcal{R}$, $\mathcal{P}$, $\mathcal{B}$ and $\mathcal{W}$ over 3000 epochs. As $\mathcal{R}$ increases $\mathcal{P}$ also increases along with it and simultaneously the average values of both $\mathcal{B}$ and $\mathcal{W}$ decreases. This confirms that the reward function we propose performs quite well to achieve our objectives. Moreover, with the aim to make *RLPM* a routing aware placement method we allow the modules to be overlapped at different time-stamps maintaining the overlapping constraint. We introduce two parameters (i) overlapping score $\widehat{O} = \left|\bigcap_{\forall i \in \widehat{B}} \mathcal{A}_i\right|$ and (ii) overlapping area $O = \left|\bigcap_{\forall e_{i,j} \in E} \mathcal{A}_i\right|$, to verify the optimized overlapping of modules and the efficient routing aware placements, respectively. For testcase T6, Fig. 6(b) presents the distribution of the average values of $\widehat{O}$, O and $\mathcal{R}$ over 3000 epochs. As $\widehat{O}$ and O increases with $\mathcal{R}$, we claim that *RLPM* is able to determine a routing-aware placement. As α increases, the constraints become less strict and the problem becomes easier for *RLPM* to solve. For testcase T6, Fig. 6(c) depicts the distributions of $\mathcal{R}$ over 3000 epochs for $\alpha = 2$ and $\alpha = 3$, respectively. Here, we observe that the learning rate increases more rapidly after a certain number of epochs when α is changed from 2 to 3.

Considering the testcase T4 we gradually increase the chip size from 18×13 to 24×19, fixing α as 2 we compare the $\mathcal{B}$ and the total cell utilization U for *HSM*, *WLSM* and *RLPM* as shown in Figs. 7(a)



(a) (b) (c)

Figure 6: For testcase T6, the average distribution of the reward with respect to (a) $\mathcal{B}$, $\mathcal{W}$ and $\mathcal{P}$ and (b) the overlap score $\widehat{O}$ and overlap area O. (c) The average distributions of the rewards for $\alpha = 2, 3$.



(a) (b) (c)

Figure 7: Comparison of (a) bounding-box area $\mathcal{B}$, (b) total cell utilization U and (c) reliability metric ρ for testcase T4 among *HSM*, *WLSM* and *RLPM* over different chip sizes, with a fixed overlap constraint $\alpha = 2$.

Table 2: Comparison of *HSM* [8], *WLSM* [15] and *RLPM* for all the testcases with sufficiently larger chip size.

Testcase	Chip size $H \times W$	*HSM* [8] U	*HSM* [8] $\mathcal{B}$	*WLSM* [15] U	*WLSM* [15] $\mathcal{B}$	*RLPM* U	*RLPM* $\mathcal{B}$
T1	28 × 28	532	624	512	572	412 **23.44%↓ | 19.53%↓**	440 (20 × 22) **29.49%↓ | 23.08%↓**
T2	34 × 34	816	960	888	1024	688 **15.69%↓ | 22.52%↓**	768 (24 × 32) **20.0%↓ | 25.0%↓**
T3	70 × 70	2764	3360	4112	4900	2500 **9.55%↓ | 39.20%↓**	2760 (46 × 60) **17.86%↓ | 43.67%↓**
T4	32 × 32	748	832	748	832	560 **25.13%↓ | 25.13%↓**	560 (20 × 28) **32.69%↓ | 32.69%↓**
T5	40 × 40	1316	1600	1344	1600	1112 **15.50%↓ | 17.26%↓**	1200 (30 × 40) **25.0%↓ | 25.0%↓**
T6	54 × 54	2092	2484	2400	2916	1564 **25.24%↓ | 34.83%↓**	1620 (30 × 54) **34.78%↓ | 44.44%↓**
T7	34 × 34	976	1156	844	1156	492 **49.59%↓ | 41.76%↓**	768 (24 × 32) **33.56%↓ | 33.56%↓**
Average improvement percentage						**23.45%**	**28.60%**	**27.63%**	**32.49%**

and 7(b), respectively. From these figures, we conclude that as the chip size increases, *RLPM* is able to find valid solutions with lesser $\mathcal{B}$ (Fig. 7(a)) and U (Fig. 7(b)) compared to the state-of-the-art methods. For a similar simulation setup, we also check the reliability metric ρ for all the methods, as shown in Fig. 7(c). In Fig. 7(c), we define reliability threshold as $\frac{H \times W}{\alpha \times \mathcal{B}}$, to compare the methods in enhancing the reliability of the chip. From this, we confirm that, on average *RLPM* produce more reliable module placement compared to *HSM* and *WLSM*. For larger chip sizes, all the methods can successfully place the modules, but *RLPM* additionally manages to reduce both U and $\mathcal{B}$. These claims are also validated in Table 2. For all the test cases, here we have considered larger chip sizes and found that both U and $\mathcal{B}$ have been drastically reduced by *RLPM*. On an average, *RLPM* minimizes U by 23.45% and 28.60%, and $\mathcal{B}$ by 27.63% and 32.49% compared to *HSM* and *WLSM*, respectively.

6 CONCLUSIONS

This paper presents a reinforcement learning based module placement method to achieve an efficient placement of rectilinear shaped modules in case of MEDA biochips. We found that the proposed module placement method can significantly increase the reliability of the MEDA biochips over the module placement results obtained by the state-of-the-art techniques while determining the smallest possible chip utilization area of the biochip for a given bioprotocol.

REFERENCES

[1] Chung et al. 2018. Module Placement Under Completion-time Uncertainty in Micro-electrode-dot-array Digital Microfluidic Biochips. *IEEE TMSC* 4, 4 (2018), 811–821.

[2] Dong et al. 2015. On the Droplet Velocity and Electrode Lifetime of Digital Microfluidics: Voltage Actuation Techniques and Comparison. *Microfluidics and Nanofluidics* 18, 4 (2015), 673–683.

[3] Einav et al. 2008. Discovery of a Hepatitis C Target and its Pharmacological Inhibitors by Microfluidic Affinity Analysis. *Nature Biotechnology* 26, 9 (2008), 1019–1027.

[4] Elfar et al. 2022. Adaptive Droplet Routing for MEDA Biochips via Deep Reinforcement Learning. In *"Proc. of the DATE"*. 640–645.

[5] Luis M. Fidalgo and Sebastian J. Maerkl. 2012. A Software-programmable Microfluidic device for automated biology. *Lab Chip* 32, 6 (2012), 353–360.

[6] Hadwen et al. 2012. Programmable Large Area Digital Microfluidic Array with Integrated Droplet Sensing for Bioassays. *Lab Chip* 12, 18 (2012), 3305–3313.

[7] Kundu et al. 2021. Mixing Models as Integer Factorization: A Key to Sample Preparation with Microfluidic Biochips. *IEEE TCAD* 41, 3 (2021), 558–570.

[8] Li et al. 2017. Droplet Size-Aware High-Level Synthesis for Micro-Electrode-Dot-Array Digital Microfluidic Biochips. *IEEE TBCAS* 11, 3 (2017), 612–626.

[9] W.H. Minhass. 2012. *System-level Modeling and Synthesis Techniques for Flow-based Microfluidic Very Large Scale Integration Biochips (PhD Thesis, IMM-PhD-2012 No. 286).* Technical University of Denmark.

[10] Mnih et al. 2013. Playing Atari with Deep Reinforcement Learning. *arXiv preprint:1312.5602* (2013).

[11] Mu et al. 2020. Microfluidic-based approaches for COVID-19 diagnosis. *Biomicrofluidics* 14, 6 (2020), 061504.

[12] Sachdeva et al. 2021. Microfluidic point-of-care testing: commercial landscape and future directions. *Front. Bioeng. Biotechnol* 8 (2021), 1537.

[13] Sista et al. 2020. Digital Microfluidic Platform to Maximize Diagnostic Tests with Low Sample Volumes from Newborns and Pediatric Patients. *Diagnostics* 10, 1 (2020), 21.

[14] Wang et al. 2011. Digital Microfluidic Operations on Micro-Electrode Dot Array Architecture. *IET Nanobiotechnol* 5, 4 (2011), 152–160.

[15] Zhong et al. 2020. Enhancing the Reliability of MEDA Biochips Using IJTAG and Wear Leveling. *IEEE TCAD* 39, 12 (2020), 2063–2076.